Epsilon Recurrent Feedback for Linear Models (ERF-LM)

Writed by

Dylan Sutton Chavez

Artificial Intelligence Engineer and Systems Engineer

© Dylan Sutton Chavez. All rights reserved.

This document is issued on the date of October 23, 2025 to whom it may concern, containing an explanation of the system ERF-LM.

Abstract

Deep learning models (e.g., Transformers, RNNs, CNNs) are increasingly deep, with training and fine-tuning cycles predominantly driven by reinforcement learning from human feedback (RLHF). As of October 23, 2025, there exists no established methodology for efficient knowledge transfer from a deep neural network to a linear model. This work proposes a framework in which a linear model quantifies its own predictive uncertainty (ε) and autonomously identifies features that require escalation to a high-capacity model. These selected features are subsequently labeled, enabling the linear model to be trained on the resulting enriched dataset, effectively bridging the gap between linear and deep representations.

1. **Introduction and Background**

Models such as the simple perceptron (Rosenblatt, 1958), linear regression (Legendre, 1805), and logistic regression (Cox, 1958) share the property of performing prediction and binary classification tasks across an arbitrary number of input features (Cover, 1958). These models operate by mapping a high-dimensional feature space to a single binary output (0, 1), providing foundational frameworks for linear decision boundaries and interpretable predictive modeling.

Linear models typically operate under the "general linear combination formula" — (Legendre, 1805). This formula can be interpreted as the dot product of the weights of each feature and their corresponding input, plus a bias term:

$$y = f(\sum_{i=1}^{n} w_i x_i + b)$$

The output of this formula is passed through a function, with each model having its own activation: simple perceptron (Step function. Heaviside, 1899), logistic regression (Sigmoid function. Verhulst, 1838). Some models, such as linear regression, do not have an activation function, as the output is a continuous value.

This method is computationally efficient, O(1), but it introduces a bias when attempting to make linear predictions in high-dimensional spaces, since the uncertainty in each prediction (y) increases with the number of dimensions (n). This, in turn, increases the probability of error as more features are added: y(n), and the current output functions of linear models cannot detect uncertainty (ε).

The ERF-LM framework incorporates an external uncertainty parameter (ε) to identify out-of-distribution or atypical examples for which the linear model has not been trained. Such instances are escalated to a high-capacity model, e.g., a large language model (LLM), which provides binary labels (0, 1) for active supervision.

The labeled data is then used to update the linear model, creating a closed-loop knowledge distillation cycle in which the linear model progressively approximates the LLM's decision boundaries. This approach achieves an effective model e≈e(LLM) while retaining extreme computational efficiency, as updates in the linear model remain O(1).

2. **System Architecture**

To implement the uncertainty mechanism (ε), the simple perceptron linear model was selected. This model converts the product of the general linear combination formula into a binary output (0, 1) using the step function:

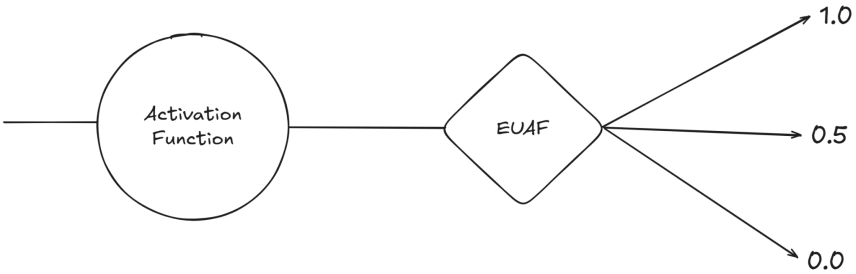
$$h(x) = 1\ (x \geq 0),\ 0\ (x < 0)$$

The uncertainty (ϵ) functions as a constant that you define before each prediction (creating a grey area). Thus, the output will be positive (1) if the continuous value (x) is greater than the uncertainty (ϵ); in the event that the output (x) is less than the negative uncertainty (−ϵ), it will be a negative prediction (0). Finally, if neither of those two conditions are false, epsilon will have been fulfilled, meaning the prediction falls into the area of uncertainty (ϵ).

$$h(x) = 1\ (x > \varepsilon),\ 0\ (x <- \varepsilon),\ 0.5\ (-\ \varepsilon \leq x \leq \varepsilon)$$

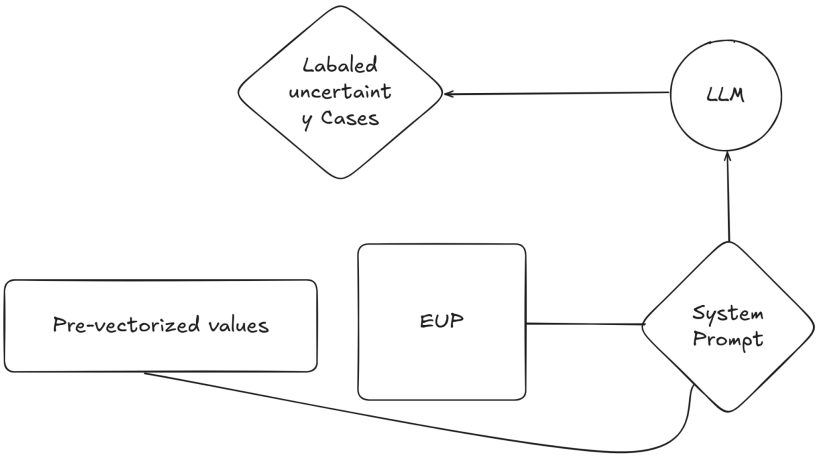
Epsilon Uncertainty Activation Function (EUAF)

The implementation of this uncertainty (ϵ) solves the problem of hallucinations in Artificial Intelligence models (the model itself knows when it doesn't know something). By detecting this and requesting help, the model receives high-confidence labels that generate high-confidence data. This prevents the model from being contaminated by attempting to re-train on examples it already knows.



Epsilon Uncertainty Perceptron (EUP)

When the model outputs uncertainty (ϵ), a request is sent to an LLM to provide the correct binary label (0, 1) for the unrecognized example. This actively labeled example is then saved to a database for use in the next fine-tuning epochs.

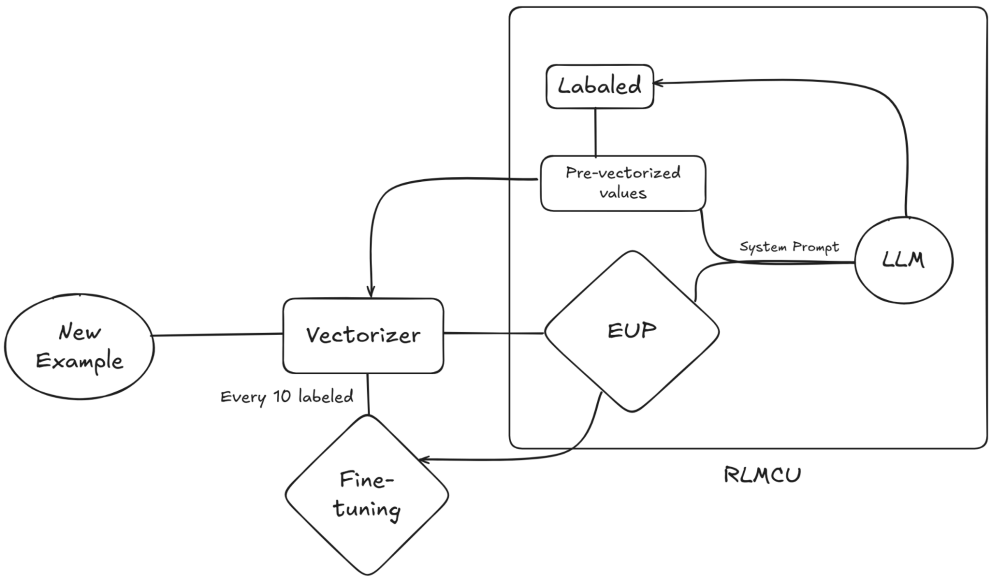


Request to the LLM Model for Cases of Uncertainty (RLMCU)

For each learning epoch, the model uses the Perceptron learning rule (Rosenblatt, 1958) to adjust its weights. This adjustment is calculated by multiplying the learning rate (η)) by the difference between the true label (y) and the perceptron's output (ŷ), based on the binary step function's output (0 or 1).

$$w_i \leftarrow w_i + \eta(y - \hat{y})x_i$$
$$b \leftarrow b + \eta(y - \hat{y})$$

The complete architecture of ERF-LM functions such that during EUP model inference, if an uncertainty case (ϵ) is detected, a call is triggered to the LLM according to the RLMCU protocol. A fine-tuning cycle is then utilized after every defined number of uncertainty (ϵ) examples, allowing the EUP model to match the efficiency of the LLM model at an O(1) computational cost.



Epsilon Recurrent Feedback for Linear Models (ERF-LM)

3. **Why Linear Models**

A wide range of tasks are inherently complex, yet ultimately exhibit linearity. Engineers typically avoid using such simple linear models, because over the decades since the advent of the first linear models, increasingly complex and deep architectures based on logic (conjunction, disjunction, etc.) have been developed. As a result, simpler linear models were largely set aside, despite their extremely low computational cost (O(1)). ERF-LM enables a simple linear model to achieve inference costs that are virtually negligible, while maintaining the efficiency of a model that would otherwise incur several dollars per inference.

4. **ERF-LM Implementation and Experimental Validation**

One of the most complex and persistent linear problems in the field of computer science is detecting whether an entity is initiating an attack (1) or generating normal traffic (0). For the validation of the ERF-LM framework, we will focus on the new OWASP Top 10: 2025 framework for web application vulnerabilities, specifically by implementing a Web Application Firewall (WAF).

LLM-Assisted Geometric Active Perceptron (L-GAP): High-Dimensional Active Learning Module for OWASP Threat Detection

The L-GAP architecture is based on ERF-LM, adding several subsystems that maintain the majority of the computational cost at O(1). This is achieved by integrating various sub-components and addressing several distinct problems.

1. **Model Input Vector:** One of the most important steps will be the definition of our vector; this is achieved by taking a security log and applying vectorization. There are several components in a security record that allow us to increase the dimensions of our vector, achieving linear separability (e.g., payload, http, metrics, metadata,...).

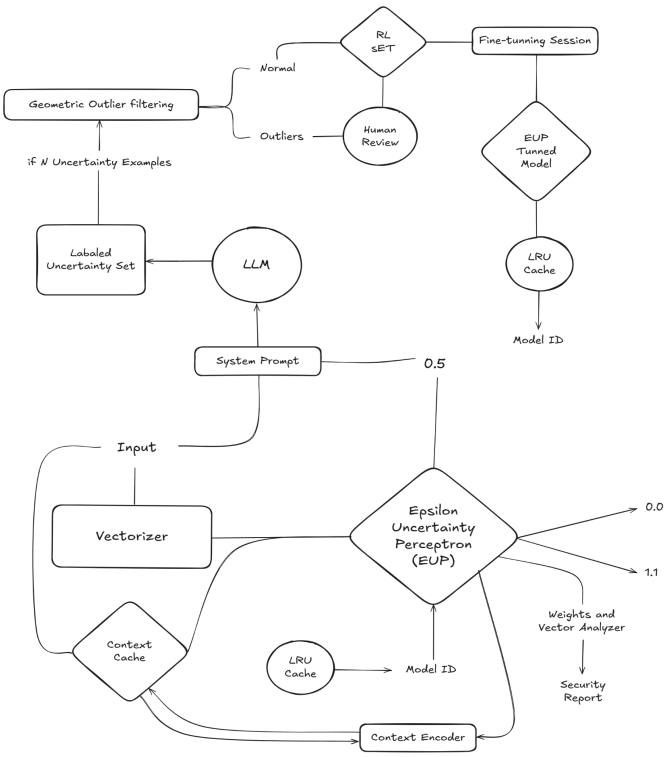
2. **Context per User:** The most sophisticated security attacks (where all current solutions fail) require the identification of complex patterns. To address this, I formulated a context per user (c), which is recalculated during each inference and dynamically adjusts the weights (w). This dynamic adjustment enables the model to effectively identify complex attacks “w(c)”.

3. **Isotropic Transformations Before the ERF-LM Fine-tuning Cycle:** The EUP model's activation function filters cases of uncertainty (ϵ), which drastically increases the prediction efficiency in a sensitive domain like cybersecurity. To incorporate an additional layer of data validation within the ERF-LM architecture, we adopt the principle: “Outlier detection employs a linear transformation... referred to as rounding” — (Dunagan, 2002). This technique leverages geometry to detect anomalous labels and patterns from the LLM, ensuring that only the most extreme geometric outliers are escalated to a human review team.

4. **Payload Vectorization with O(1) Character N-gram Hashing:** To guarantee the Perceptron's high-speed inference for payloads (e.g., SQL injections: ' OR '1'='1'), the vectorization process must operate in near O(1) time, independent of vocabulary size. This is achieved using Character N-gram Hashing. This technique bypasses the need for an external dictionary and converts the variable-length payload into a fixed-size, high-dimensional vector using a determined generation hash (salt).

5. **Weights and Vector Analyzer:** The architecture of the EUP model is so simple that we can generate a security report with each analysis by comparing the model's weights with the vector. This allows the security team to understand every decision made by the model and the reasoning behind it.

6. **Sliding Window Pre-Scanner:** In the (EUP) module, the system includes two models. The Sliding Window Pre-Scanner takes the user's payload and performs vectorization in 50-dimensional chunks using the Payload Vectorization with O(1) Character N-gram Hashing technique. This process enables a deep analysis of the user's payload to detect attacks such as SQL injection (e.g., ' OR '1'='1'). The model processes the payload using a sliding window algorithm and returns the analysis to the other EUP component, which performs a complete request analysis.



LLM-Assisted Geometric Active Perceptron (L-GAP)

4.1. **Model Input Vector**

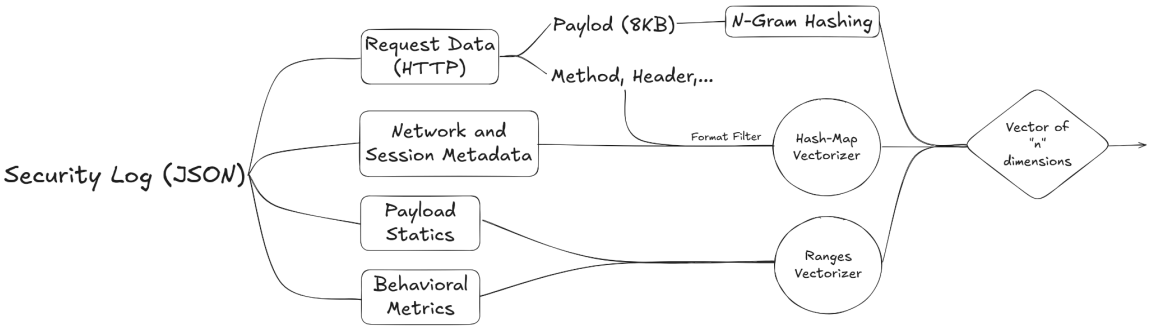
The security log applies vectorization; a security record allows us to increase the dimensions of our vector, achieving linear separability. These is based in the OWASP Application Security Verification Standards (OWASP ASVS, 2025)

1. **Request Data (HTTP): C**ontent extracted from the raw request, including the payload (query parameters; truncated at an 8KB limit to prevent DoS), the target URL, method (GET, POST,...) and headers (User-Agent,...).

2. **Network and Session Metadata:** Actor context, including the source IP address, geolocation, session,...

3. **Behavioral Metrics:** Temporal features that track the request rate per session, per errors generated, anomalies and other useful metrics.

4. **Payload Statics:** A static analysis of the payload, including its length, character entropy (detect obfuscation), the frequency of special characters,...



Security Model Vectorizer (SMV)

To transform the heterogeneous data into a unified and numerical vector, the architecture utilizes simple values to detect data with patterns (e.g., an IP range or a user agent) and Character N-gram Hashing to vectorize a payload with a number of dimensions. This process is shielded by a 384-bit (FIPS Standard) secret salt.

4.2 **Context per User**

The most deep security solutions (e.g., WAFs) and the stateless AI models fail in the high-level attacks. Some attacks like “low-and-slow”; vulnerability scanning, logic abuse, brute force attacks are impossible to detect with a single payload, the technique to detect this high-level attack is with the analysis of an actor in a long-time window.

To solve this, the L-GAP architecture introduces a stateful component: the Context Encoder. This module maintains a context vector (c) for each unique actor (e.g., IP).

$$w(c)$$

Contextual Hadamard Weight Gating

The weights are dynamically wrapped by the context vector; by multiplying each user's vector with the Hadamard method (⊙) against the weights, we achieve a new context-adjusted vector. This allows for the detection of long-term patterns.

To calculate the context of the user we need a vector that; combine past and future, and scales the vector to avoid chaos. This problem was somewhat complex, but a similar issue has existed in finance and statistics for years. This was solved using the “Exponential Moving Average (EMA)” — (Brown, 1956).

$$c_t = (1 - \alpha)c_{t-1} + \alpha v_t$$

Exponential Moving Average (EMA) — o(1)

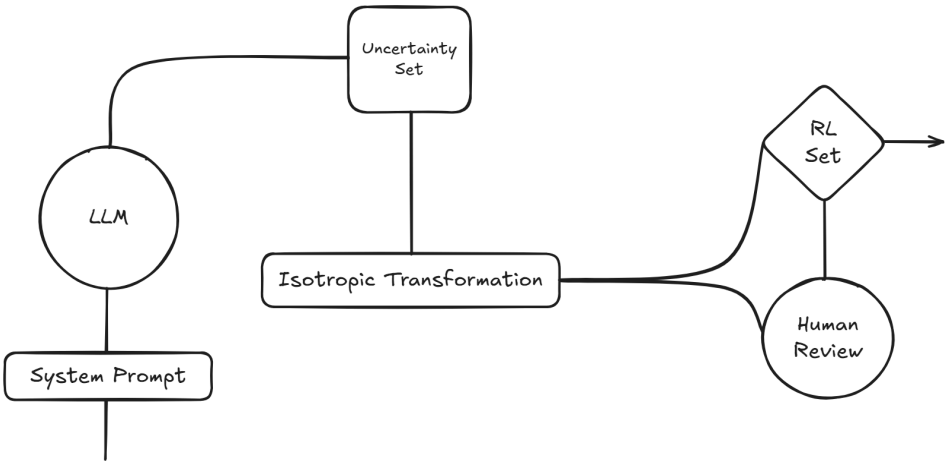
Where:

1. C_t (context vector): updated context vector
2. C_{t-1} (context vector t-1): previous context vector
3. v_t (observation vector): current comportament observation vector (current log)
4. α (alpha): exponential smoothing factor (memory decay rate)

4.3 **Isotropic Transformations Before the ERF-LM Fine-tuning Cycle**

The ERF-LM fine-tuning cycle relies on the LLM, providing high-quality labels for the “Uncertainty Set” (ε). This creates an attack vector: if an adversary hides in his payload a jailbreak, he can introduce poisoned data (OWASP, 2025) into the EUP training cycle.

To prevent this, I adopt the principle from; “In order to detect outliers, we use a linear transformation. Let M = E[xxT] where x is a sample... Consider the transformed space z = A−1x... and we will refer to the transformation as rounding.” — (John D. Dunagan, 2002, p. 21).



Isotropic Transformations Before Fine-tuning

This isotropic transformation shows the more strange labels, and these vectors are scaled to the security team for a human review (this cycle prevents data poisoning).

4.4 **Payload Vectorization with O(1) Character N-gram Hashing**

A solid security architecture needs a semantic analysis of the payload content, to verify if the payload has an attack (e.g., SQL injections: ' OR '1'='1'). While the EUP architecture's O(1) inference is a core requirement, traditional vocabulary-based vectorization represents a non-O(1).

A predictable vectorization process creates a critical attack vector: if an adversary could predict where a malicious payload will land in the vector space, they could theoretically "spoof" the isotropic transformation.

To prevent this the L-GAP architecture vectorizes the payload using n-grams (e.g., hello (n=2) → he-ll-lo), a geometrically unpredictable hashed (384-bit) projection.

$$i = H(g + salt) \quad (mod\ N)$$

Index Mapping (Hashed Projection)

$$V[i] \leftarrow V[i] + 1$$

Features Count Increment

This 384-bit cryptographic salt is the critical defensive primitive. It ensures that the resulting vector's position is computationally unpredictable to an attacker. This unpredictability is what guarantees the efficacy of the Geometric Outlier Filter; an attacker cannot "aim" their poisoned vector at the benign cluster and is forced to land in a random, isolated geometric position, ensuring their detection.

4.5 **Weights and Vector Analyzer**

One of the biggest problems in security and machine learning (ML) is the interpretability of a model, the benefit of the EUP model is the easy interpretability of the input vector (x), and the model weights (w).

The system can automatically analyze his weights (w) because each weight is mapped with his value (v) → w ∈ v (ip).

$$s = w * c * x$$

Attribution Vector Calculation

The attribution vector calculation makes the model interpretable, because the attribution (s) create a ranking of the biggest values, then these values are mapped with a map of weights and explain the decision.

4.6 **Sliding Window Pre-Scanner**

Takes the user’s payload and performs vectorization in 50-dimensional chunks using the Payload Vectorization with O(1) Character N-gram Hashing technique. This process enables a deep analysis of the user’s payload to detect attacks such as SQL injection (e.g., ' OR '1'='1'). The model processes the payload using a sliding window algorithm and returns the analysis to the other EUP component, which performs a complete request analysis.